

Recordatorio

En las iteraciones anteriores se implementó:

- **Iteración 01:** programa en consola (CLI) dentro de un único archivo, versión que sirvió entender la mecánica básica de la ruleta.
- **Iteración 02:** ejercicio de uso de ventanas para el módulo de login y registro en Swing, con usuarios hardcoded y almacenamiento en memoria.
- **Iteración 03:** aplicación del *Principio de Responsabilidad Única* (SRP), separando la lógica de la ruleta en clases.
- **Iteración 04:** incorporación del patrón *Modelo–Vista–Controlador* (MVC) y uso de getters y setters para aplicar encapsulamiento explícitamente.
- **Iteración 05:** implementar relaciones de dependencia y asociación entre clases de manera explícita a nivel de código por medio de un diagrama.
- **Iteración 06:** incorporar proceso *Round-Trip* en Visual Paradigm para el modelado e implementación de un módulo de estadística.
- **Iteración 07:** implementar el uso de herencia y polimorfismo por medio de una jerarquía de apuestas, reemplazando el flujo por if y switch's.

1. Objetivos

1.1 Objetivo General

Desacoplar la lógica del juego del historial mediante una interfaz `IRepositorioResultados`, con dos clases que implementan esta, una para la ejecución y otra que incorpore manejo en archivos para la persistencia entre sesiones.

1.2 Objetivos Específicos

1. Modelar en Visual Paradigm la interfaz `IRepositorioResultados` y las clases concretas `RepositorioEnMemoria` y `RepositorioArchivo`, junto con sus dependencias con `Ruleta` y `Estadísticas`.
2. Generar el esqueleto de código por ingeniería directa a partir del diagrama.
3. Implementar `RepositorioEnMemoria` para almacenar y consultar el historial durante la ejecución.
4. Implementar `RepositorioArchivo` para guardar y leer el historial desde un archivo (formato simple, por ejemplo CSV).
5. Integrar la interfaz en el proyecto: inyectar `IRepositorioResultados` en `Ruleta` y en el módulo de `Estadísticas`, reemplazando accesos directos a listas por llamadas al contrato.
6. Verificar el desacoplamiento ejecutando el sistema alternando entre `RepositorioEnMemoria` y `RepositorioArchivo` sin cambios en el código de `Ruleta` ni `Estadísticas`.
7. Actualizar el modelo UML mediante ingeniería inversa (reverse engineering) para reflejar el código final y mantener la trazabilidad modelo–implementación.

La motivación de esta iteración radica en otorgar al sistema robustez y flexibilidad frente a posibles fallos o pérdidas del historial, ya que con el uso de una interfaz `IRepositorioResultados`, el programa puede continuar funcionando incluso si el archivo de historial se pierde, se corrompe o no puede ser accedido por alguna razón, utilizando temporalmente el repositorio en memoria durante la sesión.

Además, esta separación entre la lógica del juego y el manejo de datos, permite la extensión del sistema en el futuro, incorporando nuevos mecanismos de persistencia como una base de datos relacional.

2. Ayudas y Sugerencias

2.1 Interfaz

Una *interfaz* define un contrato de comportamiento que una clase se compromete a implementar, el cual especifica *qué* operaciones debe ofrecer un objeto (sus métodos), sin imponer el *cómo* se implementan.

En este contexto, a diferencia de una *herencia* que modela relaciones de una jerarquía de familia para reutilizar estado y lógica común, las interfaces capturan un *comportamiento* que se repite entre clases no emparentadas entre sí.

Ejemplo:

El comportamiento `apagar()` se encuentra tanto en una clase `Auto` como en una clase `Ampolleta`, por lo que si bien no pertenecen a la misma jerarquía, ambas pueden *implementar* de diferente forma la interfaz `Apagable` y cumplir el mismo contrato.

Con esto, lo que importa acá es el *comportamiento* que se respeta en la interfaz, ya que cada clase conserva la libertad para definir su propia forma de cumplirlo.

2.2 Interfaz vs Herencia

- **Herencia (extends):** modela una relación de *familia* (A es un B), por lo que comparten *estado* y *lógica* común en la clase base.
- **Interfaz (implements):** modela *rol* (A *puede* hacer X), no hay un estado compartido; varias clases no relacionadas pueden cumplir el mismo contrato.

2.3 Java Swing

Para este punto, se ha dejado de lado, de manera intencional, el hecho de que el uso de interfaces ya ha sido utilizado dentro de las iteraciones de la Ruleta, ya que en cada clase de tipo ventana del paquete `Vista`, se hace uso de interfaces como `ActionListener` o `MouseListener`.

Demostración:

Acá se pasa un *objeto* que *implementa* `ActionListener`. Así, con la función *lambda*, el compilador crea esa instancia y la ventana depende solo de la *abstracción*, no de la clase concreta.

```
btn.addActionListener(e -> doSomething());
```

2.4 Principio de Inversión De Dependencias (DIP)

Para este punto, el uso de interfaces y herencias demuestran que el uso de abstracciones generan un código más modular, mantenible y extensible en el tiempo. Dentro de este contexto, surge el principio *Principio de Inversión de Dependencias* (DIP), el cual establece que:

«Los módulos de alto nivel y bajo nivel deben depender de abstracciones y no de implementaciones concretas. De esta forma, los detalles concretos, como de clases concretas, dependen del contrato y no al revés.»

2.4.1. Ejemplo

Una tienda necesita que su sistema para la gestión implemente la capacidad de realizar cobros considerando los distintos métodos de pago existentes.

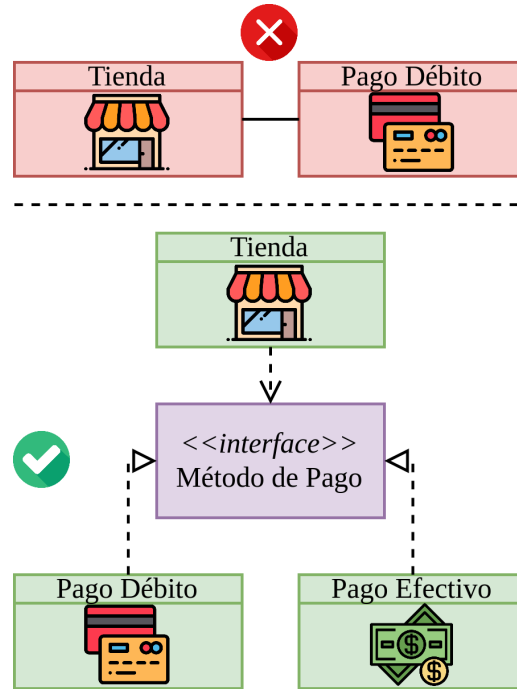


Figura 1: Enfoque Malo | Enfoque Bueno - DIP

En un primer intento, el sistema de la tienda depende directamente de una clase concreta llamada «*Pago Débito*», enfoque el cual puede parecer funcional al inicio, pero restringe al sistema a un único método de cobro, dificultando así agregar nuevos mecanismos de pago, aumentando el acoplamiento y rompiendo el principio *Open/Closed*.

En este contexto, al introducir una interfaz llamada «*Método de Pago*», la cual define las operaciones para procesar un pago, el sistema pasa a depender de una abstracción y no de una clase concreta. De este modo, cualquier método de pago nuevo que se agregue más adelante puede implementarse y funcionar sin modificar la tienda, siempre que respete el contrato definido por la interfaz; así, el mecanismo de cobro se vuelve intercambiable.

Con esto, se pasa de un enfoque basado en una implementación concreta, que vuelve al sistema rígido y con alto acoplamiento entre módulos, a un enfoque que depende de abstracciones y no de implementaciones específicas, respetando el principio *DIP*.

2. Referencias

- [1] [Documentación] Apuntes Java - Jesús. <https://jessustm.github.io/ICC490-1-P00/>
- [2] [Documentación] Apuntes List - Jesús. <https://jessustm.github.io/ICC490-1-P00/1.Apuntes/11.List/>
- [3] [Tutorial] Vincular un proyecto Java en IntelliJ IDEA con GitHub. <https://github.com/JesusTM/ICC490-1-P00/blob/main/2.Reportes/%5BRT-P00-01%5D%20GitHub%20-%20IntelliJIDEA.pdf>
- [4] [Tutorial] Cómo usar Git y GitHub. <https://github.com/JesusTM/ICC490-1-P00/blob/main/2.Reportes/%5BRT-P00-02%5D%20Uso%20de%20GitHub.pdf>
- [5] Java Round-Trip https://www.visual-paradigm.com/support/documents/vpuserguide/276/381_javaround-tr.html
- [6] [Presentación] Intro Herencia y POO. <https://campusvirtual.ufro.cl/mod/resource/view.php?id=2037372>
- [7] [Presentación] Intro Herencia (II) y POO. <https://campusvirtual.ufro.cl/mod/resource/view.php?id=2037375>
- [8] [Infografía] Principios SOLID. <https://campusvirtual.ufro.cl/mod/resource/view.php?id=2035831>
- [9] [Documentación] Apuntes Manejo de Archivos - Jesús. <https://jessustm.github.io/ICC490-1-P00/1.Apuntes/10.Manipular%20Archivos/>